

MODULE 4

Robot Framework

Module	Robot Framework
Name	Debasmita Pal
Program / Year	B.Tech CSE (IoT, CS & BT) / 4th Year
Institution	Institute of Engineering & Management, Kolkata
Enrollment No	12023002029035
Module Covered	Module 4 – Robot Framework (Experiments 4.1 – 4.3)
Date	21st September ,2026

INDEX

S. No.	Experiment / Topic	Status
4.1	Introduction to Robot Framework	Completed
4.2	Script Running Options	Completed
4.3	Make the Script More Readable	Completed

EXPERIMENT 4.1 – Introduction to Robot Framework

1. Problem Statement

To understand the basic syntax of Robot Framework and create a simple web automation test using SeleniumLibrary.

2. Objective

To set up Robot Framework, use basic keywords, open a web page, enter text and verify an element.

3. Tools, Software & Concepts Used

- Python
- Robot Framework
- SeleniumLibrary
- VS Code / PyCharm
- Web browser

Concepts: Robot Framework syntax, test cases, keywords, SeleniumLibrary, Input Text, Press Keys and Page Should Contain

4. Source Code

```
*** Settings ***
```

```
Library    SeleniumLibrary
```

```
*** Test Cases ***
```

```
Open Website And Search
```

```
    Open Browser    https://www.bing.com    chrome
```

```
    Maximize Browser Window
```

```
    Input Text    name=q    Robot Framework
```

```
    Press Keys    name=q    ENTER
```

```
    Wait Until Page Contains    Robot Framework    10s
```

```
    Page Should Contain    Robot Framework
```

```
    Sleep    2
```

```
    Close Browser
```

5. Output

5.1 TERMINAL OUTPUT

```
PS C:\Users\debas\PycharmProjects\PythonProject\mod4> robot exp4.1.robot
=====
Exp4.1
=====
Open Website And Search | PASS |
-----
Exp4.1 | PASS |
1 test, 1 passed, 0 failed
=====
Output: C:\Users\debas\PycharmProjects\PythonProject\mod4\output.xml
Log: C:\Users\debas\PycharmProjects\PythonProject\mod4\log.html
Report: C:\Users\debas\PycharmProjects\PythonProject\mod4\report.html
PS C:\Users\debas\PycharmProjects\PythonProject\mod4> 
```

Fig 4.1.1 : Robot Framework should show the executed test case and its PASS/FAIL result.

5.2 SELINIUM BROWSER OUTPUT

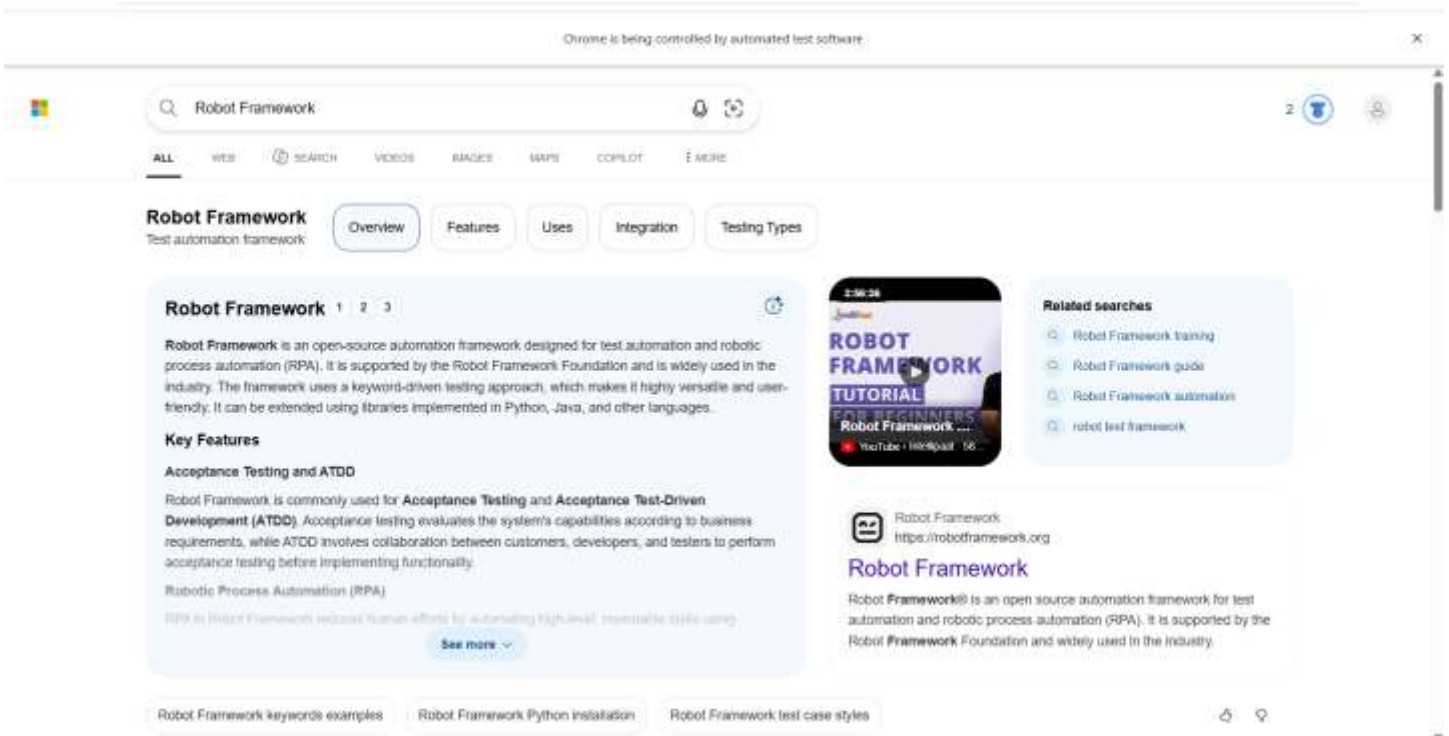


Fig 4.1.2 : The automated task completed successfully.

5.3 log.html Generated

Exp4.1 Log

Generated
20260919 23:41:25 UTC+05:30
32 seconds ago

Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	1	1	0	0	00:00:07	
Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags						
Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Exp4.1	1	1	0	0	00:00:07	

Execution Log

SUITE	Exp4.1	00:00:07.347
Full Name:	Exp4.1	
Source:	C:\Users\debse\PycharmProjects\PythonProject\mod\exp4.1.robot	
Start / End / Elapsed:	20260919 23:41:18.831 / 20260919 23:41:25.878 / 00:00:07.247	
Status:	1 test total, 1 passed, 0 failed, 0 skipped	
TEST	Open Website And Search	00:00:06.902

Fig 4.1.3: screenshot showing Experiment 4_1 log generation

5.4 report.html Generated



Fig 4.1.4: Screenshot Showing Experiment 4_1 report generation

6. Result, Observation & Conclusion

Result: Robot Framework executed the test case successfully.

Observation: The browser was opened, text was entered using a keyword and the expected text was checked on the page. Robot Framework also generated output.xml, log.html, and report.html files automatically.

Conclusion: This experiment demonstrated the basic use of Robot Framework with SeleniumLibrary for web automation. The test case was written in a .robot file using Robot Framework syntax. The successful terminal output confirmed that the complete test case executed without errors. The Selenium browser also showed that the required web actions were performed correctly. Thus, Robot Framework can be used to create simple and readable web automation test cases.

EXPERIMENT 4.2 – Script Running Options

1. Problem Statement

To understand basic ways of running Robot Framework test cases from the command line and the result files generated after execution.

2. Objective

To run a Robot Framework test from the terminal and understand setup, teardown and generated reports.

3. Tools, Software & Concepts Used

- Python
- Robot Framework
- SeleniumLibrary
- PyCharm Terminal / Command Prompt

Concepts: robot command, test setup and teardown, output.xml, log.html, report.html and running test suites

4. Source Code

```
*** Settings ***
Library    SeleniumLibrary

*** Test Cases ***
Check Google Page
    Open Browser    https://www.google.com    chrome
    Maximize Browser Window
    Title Should Be    Google
    Page Should Contain    Google
    Sleep    2
    Close Browser
```

5. Output

5.1 TERMINAL OUTPUT

```
PS C:\Users\debas\PycharmProjects\PythonProject\mod4> robot exp4.2.robot
=====
Exp4.2
=====
Check Google Page                                     | PASS |
-----
Exp4.2                                               | PASS |
1 test, 1 passed, 0 failed
=====
Output:  C:\Users\debas\PycharmProjects\PythonProject\mod4\output.xml
Log:     C:\Users\debas\PycharmProjects\PythonProject\mod4\log.html
Report:  C:\Users\debas\PycharmProjects\PythonProject\mod4\report.html

PS C:\Users\debas\PycharmProjects\PythonProject\mod4>
```

Fig 4.2.1 : Robot Framework showing the executed test case and its PASS/FAIL result.

5.2 SELINIUM BROWSER OUTPUT

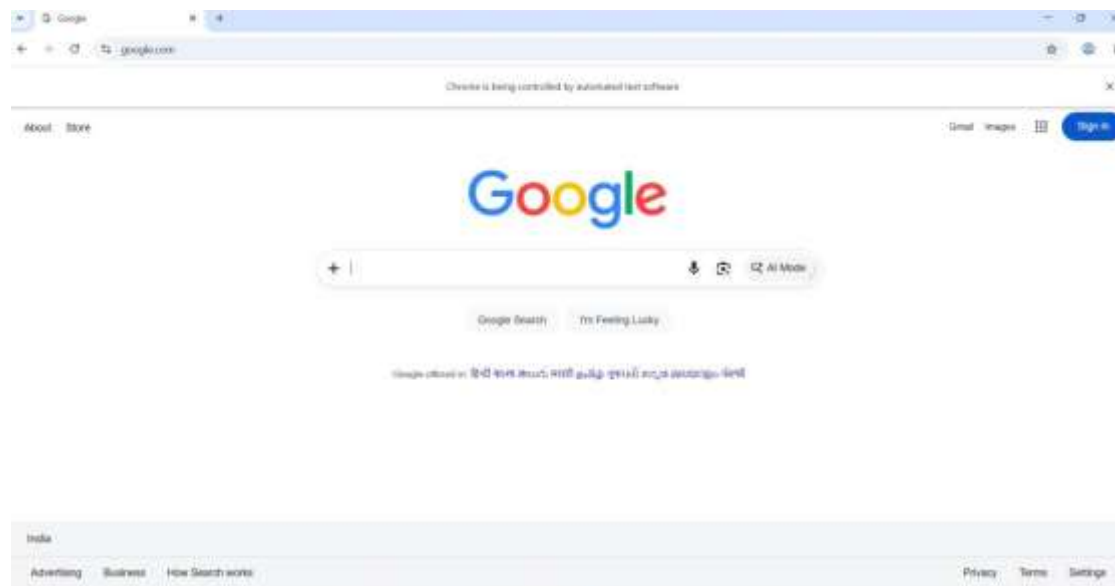


Fig 4.2.2 : The automated task completed successfully.

5.3 log.html Generated

REPORT

Experiment 4_2 Log

Generated: 20200919 02:52:13 UTC+05:30
4 minutes 0 seconds ago

Statistics

Total Statistics		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests		1	1	0	0	00:00:07	

Statistics by Tag

Statistics by Tag		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags							

Statistics by Suite

Statistics by Suite		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Experiment 4_2		1	1	0	0	00:00:07	

Execution Log

SUITE: Experiment 4_2

Full Name: Experiment 4_2

Source: C:\Users\Shamir\OneDrive\Folder\LatReport Module\Experiment_4_2.robot

Start / End / Elapsed: 20200919 02:52:06.044 / 20200919 02:52:13.233 / 00:00:07.189

Status: 1 test total, 1 passed, 0 failed, 0 skipped

+ TEST: Check Google Page

Fig 4.2.3: screenshot showing Experiment 4_2 log generation

5.4 report.html Generated



Fig 4.2.4: Screenshot Showing Experiment 4_2 report generation

6. Result, Observation & Conclusion

Result:

The Robot Framework script was executed successfully using SeleniumLibrary. The Google webpage was opened in Chrome, the page title and page content were checked, and the browser was closed successfully. The terminal showed **1 test passed and 0 failed**.

Observation:

The test case was executed using Robot Framework keywords in a simple and readable format. The Open Browser keyword opened the webpage, Title Should Be and Page Should Contain were used for verification, and Close Browser closed the browser after execution. Robot Framework also generated output.xml, log.html, and report.html files automatically.

Conclusion:

This experiment demonstrated different options for running a Robot Framework test script from the terminal. The successful terminal output confirmed that the test was executed without errors. The generated XML and HTML files provided additional execution information and test results. The experiment helped in understanding how Robot Framework runs test cases and records their execution results. Thus, the basic script-running process of Robot Framework was successfully demonstrated.

EXPERIMENT 4.3 – Make the Script More Readable

1. Problem Statement

To improve Robot Framework scripts by using user-defined keywords, resource files and setup or teardown.

2. Objective

To break a test into meaningful reusable keywords and keep common actions in a resource file.

3. Tools, Software & Concepts Used

- Python
- Robot Framework
- SeleniumLibrary
- Resource files
- VS Code / PyCharm

Concepts: Procedural vs Gherkin style, user-defined keywords, resource files, setup and teardown and reusable actions

4. Source Code

```
# resources/common.robot
*** Settings ***
Library SeleniumLibrary

*** Keywords ***
Open Test Website
    Open Browser https://www.saucedemo.com/ chrome
    Maximize Browser Window

Close Test Browser
    Close All Browsers

Login With Username And Password
    [Arguments] ${username} ${password}
    Input Text id=user-name ${username}
    Input Text id=password ${password}
    Click Button id=login-button

# Experiment_4_3.robot
*** Settings ***

Resource resources/common.robot
Suite Teardown Close Test Browser

*** Test Cases ***

Login Using Reusable Keyword
    Open Test Website

    Login With Username And Password standard_user secret_sauce

    Wait Until Page Contains Products 10 seconds

    Sleep 2
```

5. Output

5.1 TERMINAL OUTPUT

```
PS C:\Users\debas\PycharmProjects\PythonProject\mod4> robot exp4.3.robot
=====
Exp4.3
=====
Login Using Reusable Keyword | PASS |
-----
Exp4.3 | PASS |
1 test, 1 passed, 0 failed
=====
Output: C:\Users\debas\PycharmProjects\PythonProject\mod4\output.xml
Log: C:\Users\debas\PycharmProjects\PythonProject\mod4\log.html
Report: C:\Users\debas\PycharmProjects\PythonProject\mod4\report.html
PS C:\Users\debas\PycharmProjects\PythonProject\mod4> |
```

Fig 4.3.1 : Robot Framework showing the executed test case and its PASS/FAIL result.

5.2 ROBOT FRAMEWORK / BROWSER OUTPUT

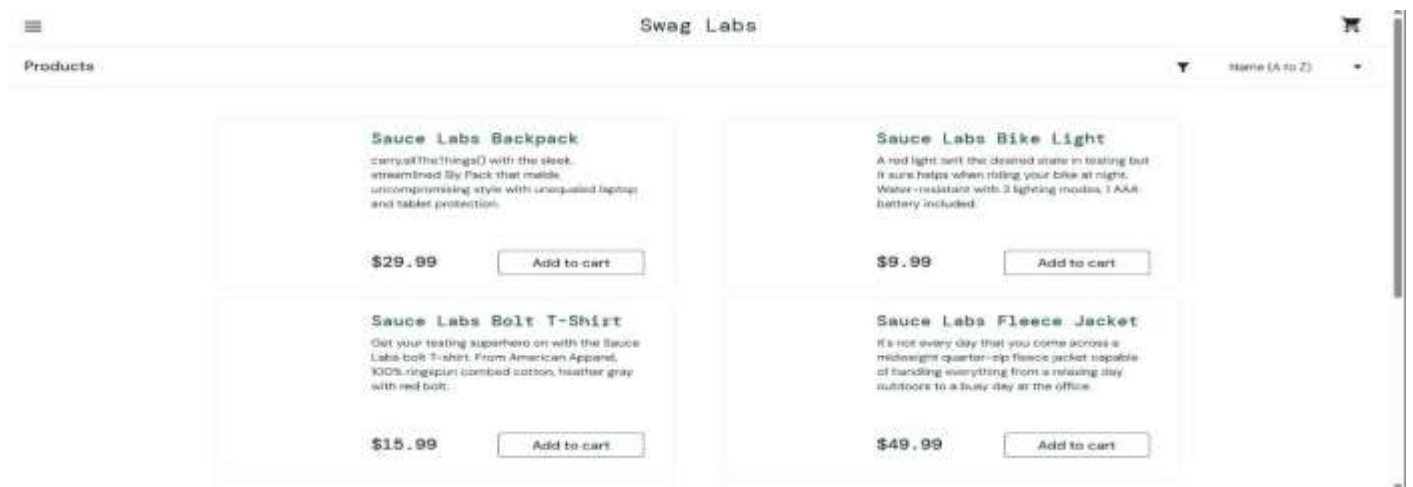


Fig 4.3.2: The automated task completed successfully.

5.3 log.html Generated



Fig 4.2.3: screenshot showing Experiment 4_3 log generation

5.4 report.html Generated



Fig 4.2.4: Screenshot Showing Experiment 4_2 report generation

5.5 Files Generated After Execution

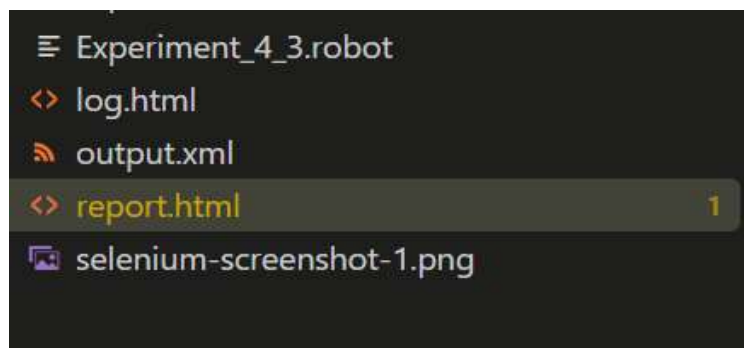


Fig 4.2.5 : Screenshot showing output.xml , selenium-screenshot-1.png , log.html and report.html generated

6. Result, Observation & Conclusion

Result:

The Robot Framework test was executed successfully using a reusable resource file and SeleniumLibrary. The browser was opened, the SauceDemo login was performed using a reusable keyword, and the Products page was verified successfully. The terminal showed **1 test passed and 0 failed**.

Observation:

The common Selenium operations were placed in common.robot and reused in Experiment_4_3.robot. Custom keywords such as Open Test Website, Login With Username And Password, and Close Test Browser made the test case shorter and easier to understand. The Wait Until Page Contains keyword was also used to wait for the Products page before verification.

Conclusion:

This experiment demonstrated how Robot Framework scripts can be made more readable and reusable using resource files and custom keywords. Instead of writing all Selenium commands directly in the test case, common operations were separated into `common.robot`. The successful terminal output confirmed that the reusable keywords worked correctly. The experiment also showed how reusable components can make automation scripts easier to maintain and understand. Thus, the objective of making the Robot Framework script more readable was successfully achieved.